✅ Complete File Checklist

Use this checklist to ensure all files are in place before running the application.

📋 Core Application Files (5 files)

1. ✅ app.py

Location: Root directory

Size: ~400 lines

Purpose: Main Flask application with all API routes

Key Functions:

- `/` - Serve main page
- `/api/upload-cv` - Handle CV upload
- `/api/search-jobs/<user_id>` - Search and match jobs
- `/api/jobs/<user_id>` - Get user's jobs
- `/api/mark-applied/<job_id>` - Mark job as applied
- `/api/notifications/<user_id>` - Get notifications

Verify: `ls -la app.py` or `dir app.py`

---

2. ✅ requirements.txt

Location: Root directory

Size: 23 dependencies

Purpose: Python package dependencies

Key Packages:

- Flask, Flask-CORS, Flask-SQLAlchemy
- PyPDF2, python-docx, pdfplumber
- spacy, beautifulsoup4, requests
- APScheduler, python-dotenv

Verify: `cat requirements.txt` or `type requirements.txt`

---

3. ✅ .env

Location: Root directory

Purpose: Environment variables and API keys

Required Variables:

```bash
SECRET_KEY=          # Required
FLASK_ENV=           # Required (development/production)
REMOTEOK_API_ENABLED=true  # Works without API key!
```

Optional Variables:

```bash
ADZUNA_APP_ID=       # For more jobs
ADZUNA_APP_KEY=      # For more jobs
THEMUSE_API_KEY=     # For more jobs
MAIL_USERNAME=       # For email notifications
MAIL_PASSWORD=       # For email notifications
```

Create: See API_SETUP_GUIDE.md

Verify: `cat .env` (check it exists, don't share!)

---

4. ✅ .gitignore

Location: Root directory

Purpose: Prevent sensitive files from being committed to Git

Protects:

- .env file (API keys!)
- **pycache**/ (Python bytecode)
- *.db (database files with personal data)
- static/uploads/ (CV files with personal data)
- venv/ (virtual environment)

Verify: `cat .gitignore`

---

5. ✅ test_apis.py

Location: Root directory

Size: ~200 lines

Purpose: Test all configured API keys

Tests:

- Adzuna API connection
- The Muse API connection
- RemoteOK API connection
- Reed API connection
- RapidAPI (JSearch) connection
- Email configuration

Run: `python test_apis.py`
Expected: Shows which APIs are working

🔧 Backend Module Files (5 files)

6. ✅ modules/init.py

Location: `modules/` directory
Size: Empty file (0 bytes)
Purpose: Make modules directory a Python package

Create: `touch modules/__init__.py` (Linux/Mac) or `type nul > modules\__init__.py` (Windows)

---

7. ✅ modules/cv_parser.py

Location: `modules/` directory
Size: ~350 lines
Purpose: Extract information from CV files
Key Functions:

- `extract_text_from_pdf()` - Parse PDF files
- `extract_text_from_docx()` - Parse DOCX files
- `extract_email()` - Find email address
- `extract_phone()` - Find phone number
- `extract_name()` - Extract name using spaCy
- `extract_skills()` - Find skills
- `extract_experience()` - Calculate years of experience
- `extract_education()` - Find education details
- `detect_country()` - Determine country from CV
- `parse_cv()` - Main parsing function

Verify: File should import PyPDF2, docx, spaCy

---

8. ✅ modules/job_scraper.py

Location: `modules/` directory

Size: ~500 lines

Purpose: Scrape jobs from multiple sources

Job Sources:

- Adzuna API (Bangladesh, India, UK, US, etc.)
- The Muse API (professional jobs)
- RemoteOK API (remote jobs)
- Reed API (UK jobs)
- JSearch API via RapidAPI (global)
- BDJobs.com (web scraping)
- Chakri.com (web scraping)
- Prothom Alo (web scraping)

Key Functions:

- `scrape_adzuna_api()` - API scraping
- `scrape_themuse_api()` - API scraping
- `scrape_remoteok_api()` - API scraping
- `scrape_reed_api()` - API scraping
- `scrape_jsearch_rapidapi()` - API scraping
- `scrape_bdjobs()` - Web scraping
- `scrape_chakri_com()` - Web scraping
- `scrape_all_jobs_with_apis()` - Main aggregator

Verify: File should import requests, BeautifulSoup, dotenv

---

9. ✅ modules/matcher.py

Location: `modules/` directory

Size: ~250 lines

Purpose: Match CV with jobs and calculate scores

Key Functions:

- `calculate_match_score()` - Calculate 0-100 score
- `match_skills()` - Skills similarity
- `match_experience()` - Experience level match
- `match_location()` - Location/country match
- `filter_applicable_jobs()` - Filter by minimum score
- `categorize_jobs()` - Group by match quality

Algorithm:

```
Score = Skills (50%) + Experience (30%) + Location (20%)
```

Verify: File contains scoring logic

---

10. ✅ modules/notifier.py

Location: `modules/` directory
Size: ~200 lines
Purpose: Deadline notifications and reminders
Key Functions:

- `schedule_deadline_checks()` - Run every 6 hours
- `check_deadlines()` - Find upcoming deadlines
- `create_notification()` - Create notification record
- `send_email_notification()` - Send email (if configured)
- `get_pending_notifications()` - Get user notifications
- `get_upcoming_deadlines()` - Get jobs with upcoming deadlines

Features:

- Background scheduler
- Browser notifications
- Email notifications (optional)
- 3-day advance warning

Verify: File should import APScheduler, Flask-Mail

💾 Database Model Files (2 files)

11. ✅ models/init.py

Location: `models/` directory

Size: Empty file (0 bytes)

Purpose: Make models directory a Python package

Create: `touch models/__init__.py` or `type nul > models\__init__.py`

---

12. ✅ models/database.py

Location: `models/` directory

Size: ~100 lines

Purpose: Database models and relationships

Models:

User Model:

- id, name, email, phone
- country, experience_years
- skills (JSON), education
- cv_filename, created_at

Job Model:

- id, user_id (FK)
- title, company, location
- job_url, deadline, source
- requirements, match_score
- is_applied, is_notified

Notification Model:

- id, user_id (FK), job_id (FK)
- notification_date, message
- is_sent, created_at

Verify: File should import SQLAlchemy

🎨 Frontend Files (3 files)

13. ✅ templates/index.html

Location: `templates/` directory

Size: ~150 lines

Purpose: Main web interface

Sections:

1. Header with title
2. Upload CV section
3. CV information display
4. Job search results
5. Notification panel

Features:

- Responsive design
- Three-step workflow
- File upload interface
- Job cards with match badges
- Filter tabs

Verify: File should be valid HTML5

---

14. ✅ static/css/style.css

Location: `static/css/` directory

Size: ~600 lines

Purpose: Complete styling for application

Styles:

- Modern gradient backgrounds
- Responsive layout (mobile-friendly)
- Card-based design
- Color-coded match badges
- Smooth animations and transitions
- Notification panel styling

Color Scheme:

- Primary: Purple gradient (`#667eea` → `#764ba2`)
- Success: Green (`#28a745`)
- Warning: Yellow (`#ffc107`)
- Danger: Red (`#dc3545`)

Verify: File should contain CSS classes

---

15. ✅ static/js/main.js

Location: `static/js/` directory
Size: ~400 lines
Purpose: Frontend JavaScript logic
Key Functions:

- `handleFileUpload()` - CV upload handler
- `displayCVData()` - Show parsed CV info
- `searchJobs()` - Trigger job search
- `displayJobs()` - Render job cards
- `createJobCard()` - Generate job HTML
- `filterJobs()` - Filter by match quality
- `markAsApplied()` - Mark job as applied
- `checkNotifications()` - Check for deadline alerts
- `showBrowserNotification()` - Display browser notification

Features:

- Async/await for API calls
- Error handling
- Browser notifications API
- Dynamic UI updates
- Filter functionality

Verify: File should contain JavaScript functions

📚 Documentation Files (6 files)

16. ✅ README.md

Location: Root directory

Size: ~800 lines

Purpose: Main project documentation

Contents:

- Project overview
- Feature list
- Quick start guide
- API information
- Configuration guide
- Testing instructions
- Deployment options
- Troubleshooting
- Security practices

Verify: Should explain entire project

---

17. ✅ QUICK_START.md

Location: Root directory

Size: ~200 lines

Purpose: Get running in 5 minutes

Steps:

1. Install dependencies
2. Configure .env
3. Run application
4. Upload CV
5. Browse jobs

Verify: Should be beginner-friendly

---

18. ✅ API_SETUP_GUIDE.md

Location: Root directory

Size: ~600 lines

Purpose: Detailed API key setup instructions

Covers:

- Adzuna API signup
- The Muse API signup
- RemoteOK (no signup!)
- Reed API signup
- RapidAPI (JSearch) signup
- Email configuration
- ngrok setup

Verify: Should have step-by-step instructions for each API

---

19. ✅ SETUP_INSTRUCTIONS.md

Location: Root directory
Size: ~500 lines
Purpose: Complete setup and deployment guide
Covers:

- Prerequisites
- Installation steps
- Configuration
- Global access (ngrok)
- Cloud deployment
- Troubleshooting

Verify: Should cover deployment scenarios

---

20. ✅ PROJECT_SUMMARY.md

Location: Root directory
Size: ~800 lines
Purpose: Complete project overview
Covers:

- All files explained
- Features implemented
- Technologies used
- Algorithms explained
- Performance metrics
- Future roadmap

Verify: Should summarize everything

---

21. ✅ FILE_CHECKLIST.md

Location: Root directory
Purpose: This file! Complete file list

🛠️ Installation Scripts (2 files)

22. ✅ install.sh

Location: Root directory
Purpose: Automated setup for Linux/Mac
What it does:

1. Checks Python installation
2. Creates virtual environment
3. Installs dependencies
4. Downloads spaCy model
5. Creates directories
6. Creates .env file
7. Tests installation

Run: `chmod +x install.sh && ./install.sh`

---

23. ✅ install.bat

Location: Root directory
Purpose: Automated setup for Windows
What it does:

1. Checks Python installation
2. Creates virtual environment
3. Installs dependencies
4. Downloads spaCy model
5. Creates directories
6. Creates .env file
7. Tests installation

Run: `install.bat`

📁 Directories to Create

24. ✅ static/uploads/

Purpose: Store uploaded CV files
Create: `mkdir -p static/uploads`
Note: Add `.gitkeep` to track in Git but ignore contents

---

25. ✅ data/

Purpose: Store SQLite database
Create: `mkdir -p data`
Auto-created: Database file created on first run

🎯 Final Verification

Run these commands to verify everything:

```bash
# Check all Python files exist
ls -la *.py
ls -la modules/*.py
ls -la models/*.py

# Check frontend files
ls -la templates/*.html
ls -la static/css/*.css
ls -la static/js/*.js

# Check documentation
ls -la *.md

# Check configuration
ls -la .env .gitignore

# Test installation
python test_apis.py
```

✅ Quick Setup Commands

For Linux/Mac:

```bash
# One-command setup
chmod +x install.sh && ./install.sh

# Or manual:
pip install -r requirements.txt
python -m spacy download en_core_web_sm
touch modules/__init__.py models/__init__.py
mkdir -p static/uploads data
python app.py
```

For Windows:

```batch
REM One-command setup
install.bat

REM Or manual:
pip install -r requirements.txt
python -m spacy download en_core_web_sm
type nul > modules\__init__.py
type nul > models\__init__.py
mkdir static\uploads data
python app.py
```

🎉 Completion Checklist

Before running the application, ensure:

- [ ] All 21+ files are present
- [ ] `modules/__init__.py` exists
- [ ] `models/__init__.py` exists
- [ ] `.env` file is configured
- [ ] Dependencies installed (`pip install -r requirements.txt`)
- [ ] spaCy model downloaded (`python -m spacy download en_core_web_sm`)
- [ ] Directories created (`static/uploads`, `data`)
- [ ] APIs tested (`python test_apis.py`)
- [ ] Application runs (`python app.py`)

Total Files: 25 items (21 files + 4 auto-created)

Ready to launch! 🚀

Run: `python app.py` and open `http://localhost:5000`